QUIZ

QUIZ

How many widgets are present in the following code?

```
void main() {
  runApp(
    MaterialApp(
        home: Center(
           child: Container(
                padding: const EdgeInsets.all(20),
                color: Colors.red,
                child: Container(color: Colors.green, width: 30, height: 30),
                ),
        )),
  );
}
```

A. 4 widgets
B. 5 widgets
C. 6 widgets
D. 7 widgets

CORRECTION

How many widgets are present in the following code?

```
void main() {
  runApp(
    MaterialApp(
        home: Center(
            child: Container(
                padding: const EdgeInsets.all(20),
                color: Colors.red,
                child: Container(color: Colors.green, width: 30, height: 30),
                ),
        )),
  );
}
```

A. 4 widgets
B. 5 widgets
C. 6 widgets
D. 7 widgets

QUIZ

Match each widget with the correct description

A. Text

B. Container

C. Row

D. Column

1. Arranges widgets vertically
2. Displays a string of text
3. Arranges widgets horizontally
4. A box used for layout, styling, padding..

CORRECTION

Match each widget with the correct description



QUIZ

Match Properties with Widgets

A. Row

B. Column

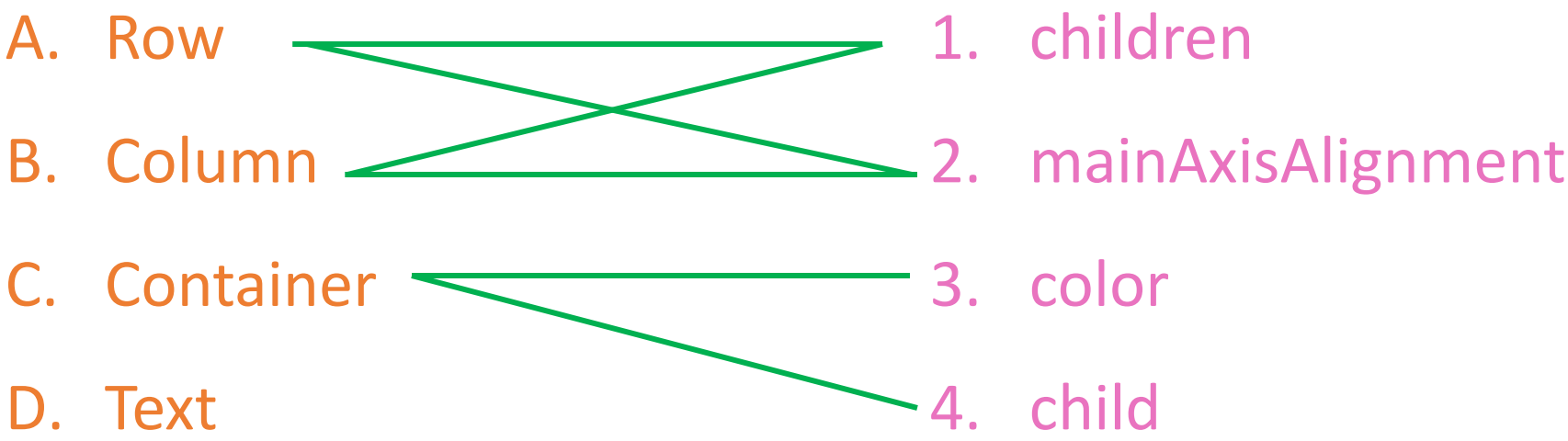
C. Container

D. Text

1. children
2. mainAxisAlignment
3. color
4. child

CORRECTION

Match Properties with Widgets



QUIZ

Which of the following instantiations are valid? Select all that apply:

```
MyWidget(a, [b], [c = 10], { d, required e, f = 10 }) { … }
```

1. `MyWidget(1, 2, 3, e: 5)`
2. `MyWidget(1, e: 5)`
3. `MyWidget(1, 2, 3, d: 4, f: 6)`
4. `MyWidget(1, e: 5, d: 6)`
5. `MyWidget(a: 1, e: 5)`

CORRECTION

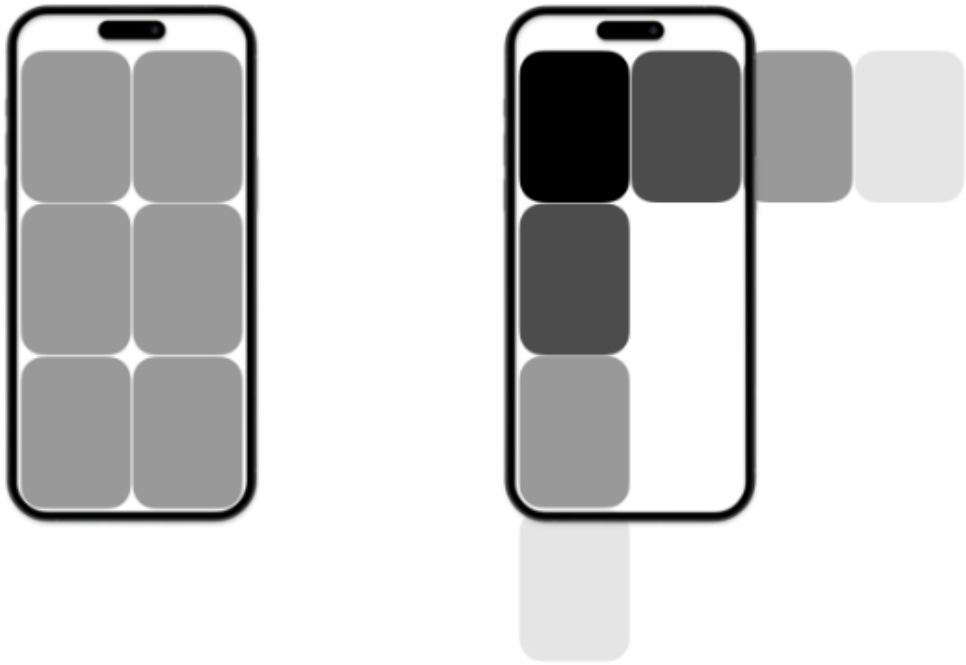
Which of the following instantiations are valid? Select all that apply:

```
MyWidget(a, [b], [c = 10], { d, required e, f = 10 }) { … }
```

1. `MyWidget(1, 2, 3, e: 5)`
2. `MyWidget(1, e: 5)`
3. `MyWidget(1, 2, 3, d: 4, f: 6)` e is required
4. `MyWidget(1, e: 5, d: 6)`
5. `MyWidget(a: 1, e: 5)` A is positional, cannot be named

MOBILE DEVELOPMENT

W5 – Layouting



CADT IDT

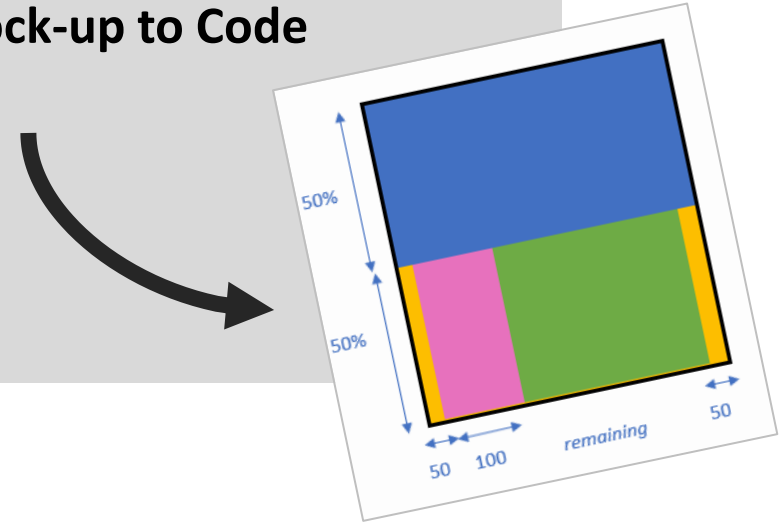
🎯 THIS WEEK =**PREPARATION FOR NEXT WEEK**

QUIZ!

- ✓ Basic **Widgets Usage**
- ✓ Composition of widgets
- ✓ **Layouting Rules**

TEST!

- ✓ Basic Widgets Usage
- ✓ From **Mock-up to Code**





Course Objectives



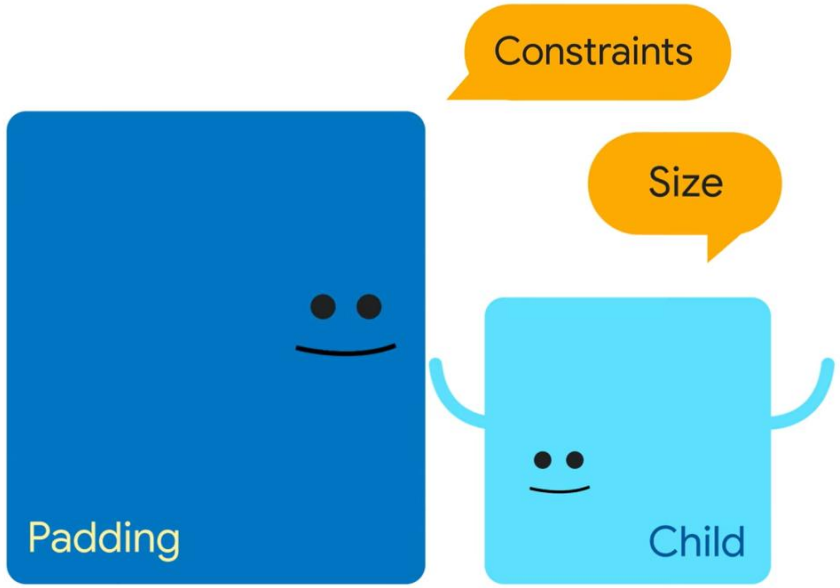
- ✓ **Understand Flutter Layouting**
  - Combine `Container` + `Row` + `Column` + `Expanded`
  - Understand widget Layouting rules
    `Constraints` goes down; `sizes` goes up
- ✓ **Apply a Layout Workflow**
  - Build layouts from top to bottom.
- ✓ **Solve Layout errors**
  - Understand unbounded height issues.
  - Use `Expanded` and `SingleChildScrollView` appropriately.

ACTIVITY

Flutter **layouting**

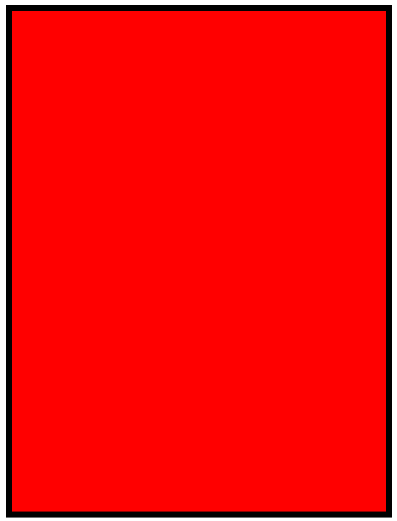
For each of the 15 provided codes:

- ✓ **Observe** — Read the code and examine the rendered screen
- ✓ **Investigate** — Consult the documentation
  Search online or use AI to understand the layout behavior. (Flutter Layout Constraints)
- ✓ **Explain**: — Why the widgets are rendered this way, *focusing on constraints, sizing, and alignment*.



DEMO CASE 1

```
Container(color: red)
```

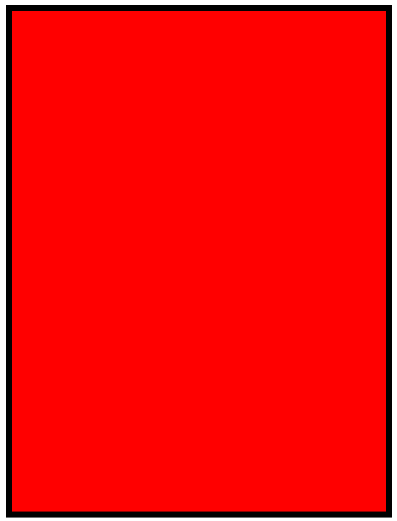


The screen is the parent of the Container,

it **forces the Container** to be exactly the same size as the screen.

DEMO CASE 2

```
Container(
          width: 100,
          height: 100,
          color: red)
```

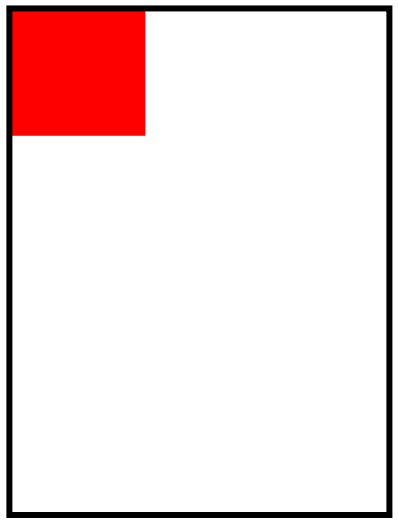


The red Container **wants to be 100 × 100**

But it can’t
Because the screen **forces it to be exactly the same size as the screen.**

DEMO CASE 3

```
Scaffold(
 child: Container(
          width: 100,
          height: 100,
          color: red),)
```

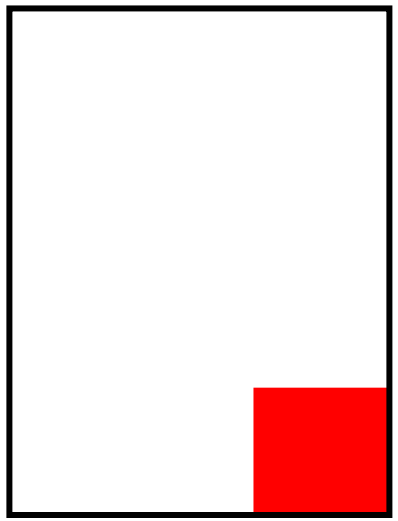


The Scaffold tells the Container can **be any size it wants**

Now the Container can **indeed be 100 × 100**.

DEMO CASE 4

```
Align(
  alignment: Alignment.bottomRight,
  child: Container(
          width: 100,
          height: 100,
          color: red),)
```

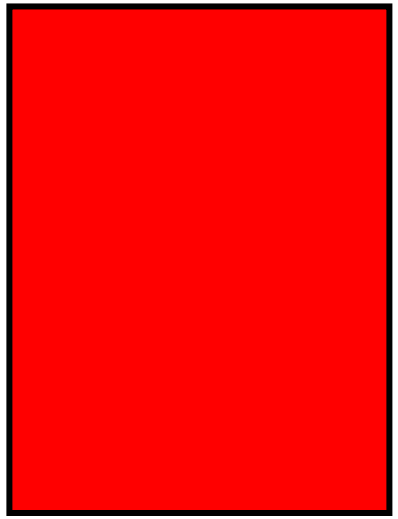


Align tells the Container that it can **be any size it wants**

It aligns the **container to the bottom-right of the available space.**

DEMO CASE 5

```
Center(
  child: Container(
         width: double.infinity,
         height: double.infinity,
         color: red),)
```



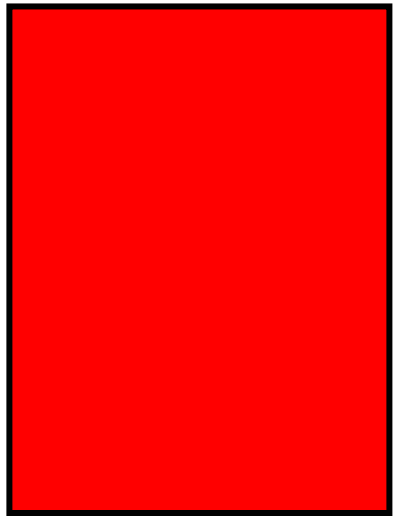
The Center tells the Container that it can be any size it wants, **but not bigger than the screen**.

The Container **wants to be of infinite size,**

But since it can't be bigger than the screen, it just fills the screen.

DEMO CASE 6

```
Center(
    child: Container(color: red),)
```

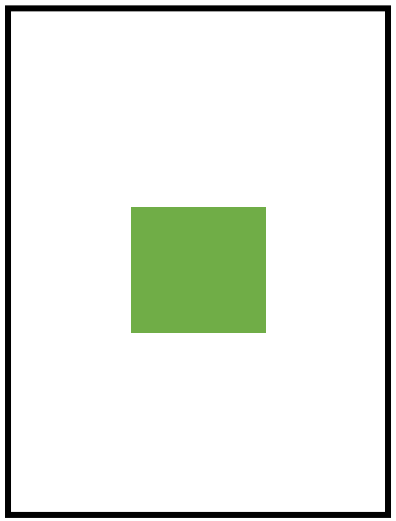


The Center tells the Container that it **can be any size it wants**

Since the Container has no child and no fixed size, **it decides it wants to be as big as possible, so it fills the whole screen.**

DEMO CASE 7

```
Center(
  child: Container(
    color: red,
    child: Container(
          color: green,
          width: 30, height: 30),
), )
```

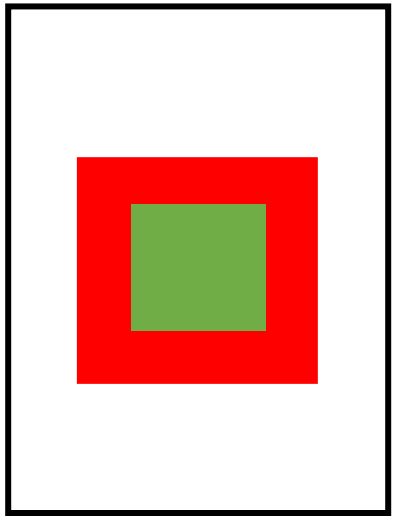


Since the red Container has no size but has a child,

it decides **it wants to be the same size as its child.**

DEMO CASE 8

```
Center(
  child: Container(
    padding: const EdgeInsets.all(20),
    color: red,
    child: Container(
         color: green,
         width: 30, height: 30), ), )
```

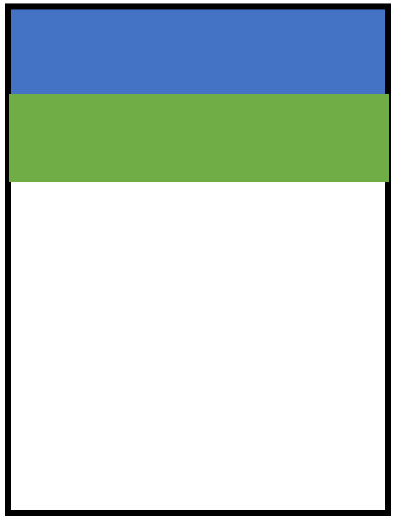


Container sizes itself to its children's size,

but it also takes its own padding into consideration

DEMO CASE 9

```
Column(
   children: [
      Container(height: 100, color:blue),
      Container(height: 100, color:green)
    ],),
```



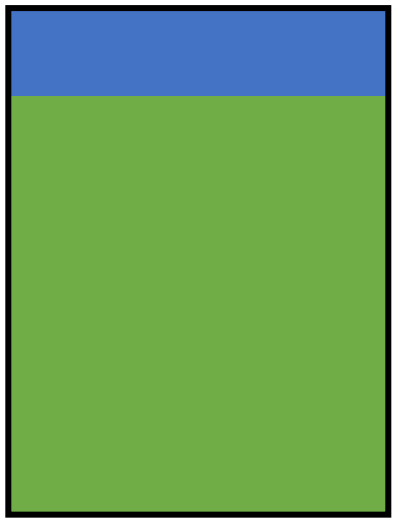
The Column arranges its 2 children vertically.

The container will **stretch to fill the available width.**

DEMO CASE 10

```
Column(
 children: [
    Container(height: 100, color:blue),

    Expanded(
       child:
         Container(height: 100, color:green)
     )],),
```



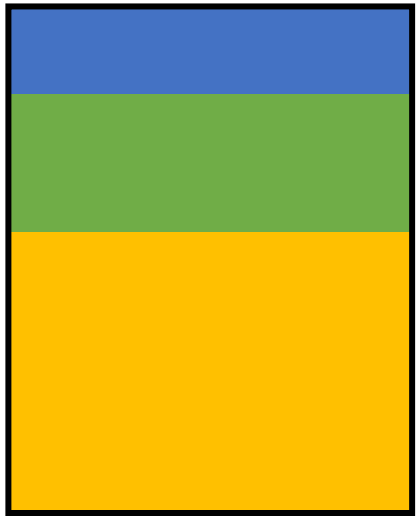
The Expanded widget expands a child of a Row, Column,

The green container fill the available space

DEMO CASE 11

```
Column(
        children: [
          Container(
           height: 100,
            color: Colors.blue,

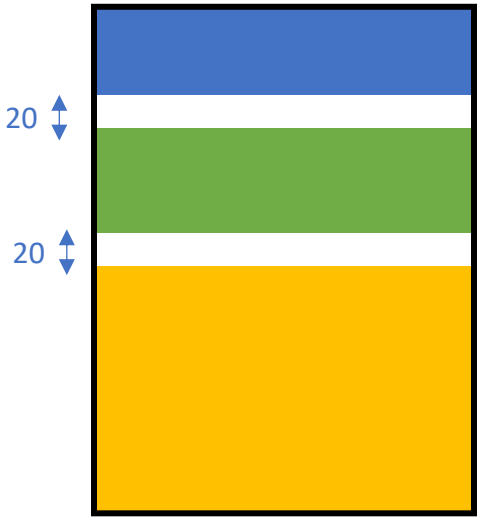
          ),
            Expanded(
              flex: 1,
              child: Container(
                          height: 100,
                          color: Colors.green,
                         ),
            ),
            Expanded(
              flex: 2,
              child: Container(
                          height: 100,
                          color: Colors.yellow,
                         ),
            ),
        ],
      ),
```



The green container has **expanded with a flex 1**

The yellow container has **expanded with a flex 2**

DEMO CASE 12

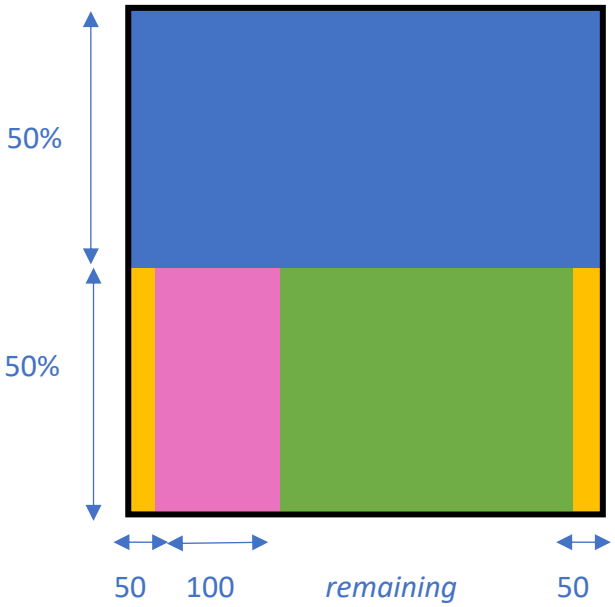


A vertical space is added between the containers

```
Column(
        children: [
          Container(
           height: 100,
            color: Colors.blue,

          ),
           const SizedBox(height: 20,),
           Expanded(
             flex: 1,
             child: Container(
                        height: 100,
                        color: Colors.green,
                       ),
           ),
            const SizedBox(height: 20,),
           Expanded(
             flex: 2,
             child: Container(
                        height: 100,
                        color: Colors.yellow,
                       ),
           ),
        ],
      ),
```

DEMO CASE 13



50 % each

Left and right padding

Nested containers

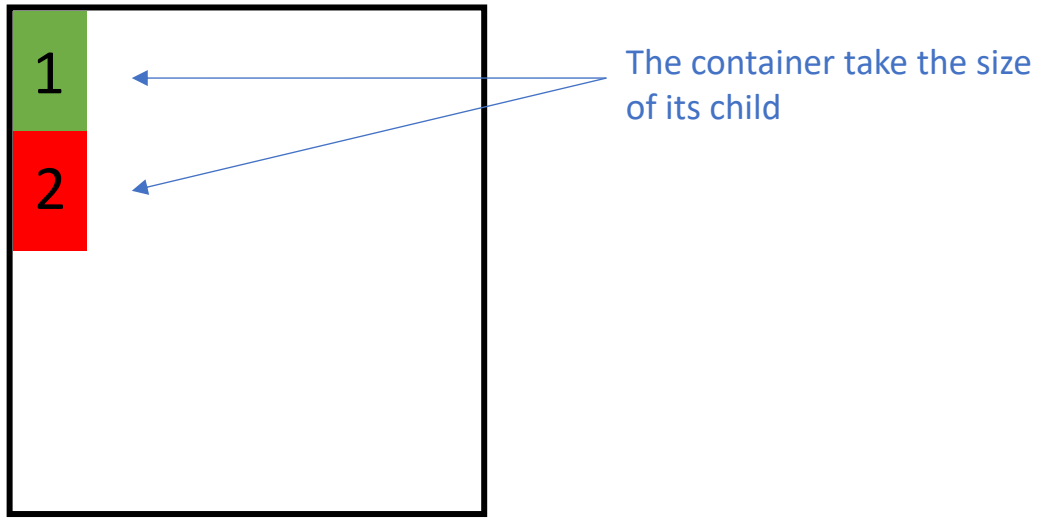
The green takes the remaining space

```
Column(children: [
      Expanded(
        child: Container(
          color: Colors.blue,
        ),
      ),
    Expanded(
        child: Container(
          padding: const EdgeInsets.only(left: 50, right: 50),
          color: Colors.yellow,
          child: Row(
            children: [
              Container(width:100, color: Colors.pink),
              Expanded(child: Container(color: Colors.green)),
            ],

          ),
      ),
    ),
  ])),
```

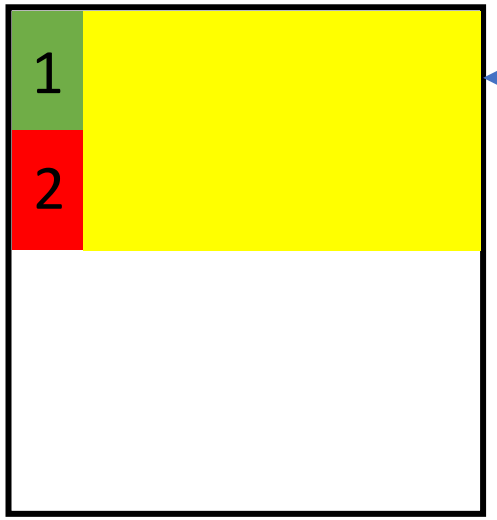
DEMO CASE 14

```
Column(
    children: [
            Container(
                color: Colors.yellow,
                child: const Text("1",
                  style: TextStyle(backgroundColor: Colors.red),
                )),
            Container(
                color: Colors.yellow,
                child: const Text("2",
                   style: TextStyle(backgroundColor: Colors.green),
                )),
          ],
)
```



DEMO CASE 15

```
Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
            Container(
                color: Colors.yellow,
                child: const Text("1",
                  style: TextStyle(backgroundColor: Colors.red),
                )),
            Container(
                color: Colors.yellow,
                child: const Text("2",
                   style: TextStyle(backgroundColor: Colors.green),
                )),
          ],
)
```



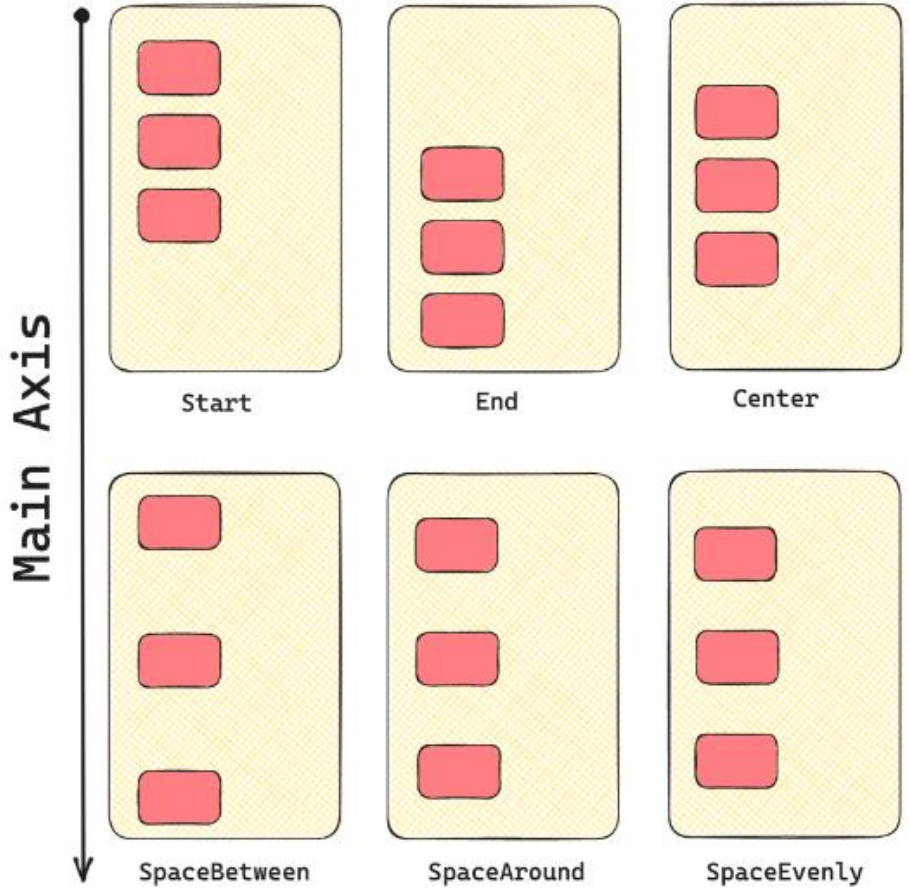
The column **constraints** the yellow container width

EXPLAIN

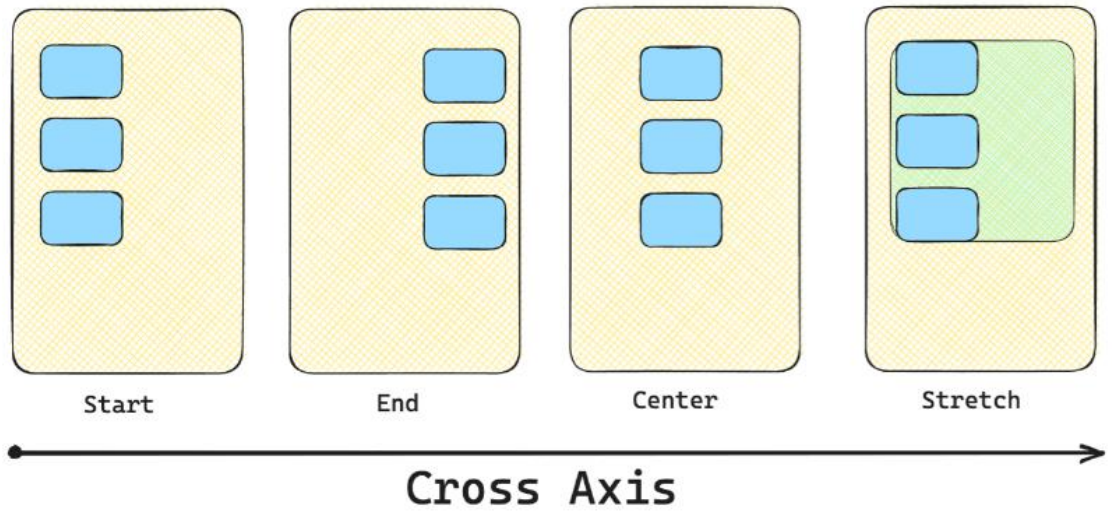
Column & Row: main/cross axis

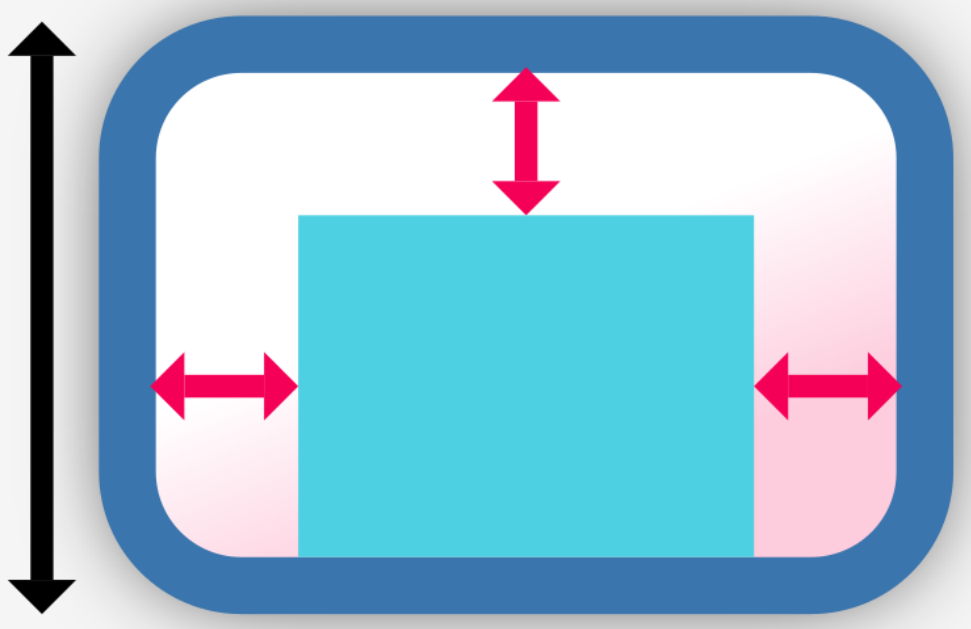
Same behavior as CSS flex

MORE INFO



The column **constraints** the children width





“Constraints go down.
Sizes go up.
Parent sets position.”

EXPLAIN

Widget **Size Constraints**

Widgets get sized based on : their **own** preferences + their **parent** size constraints

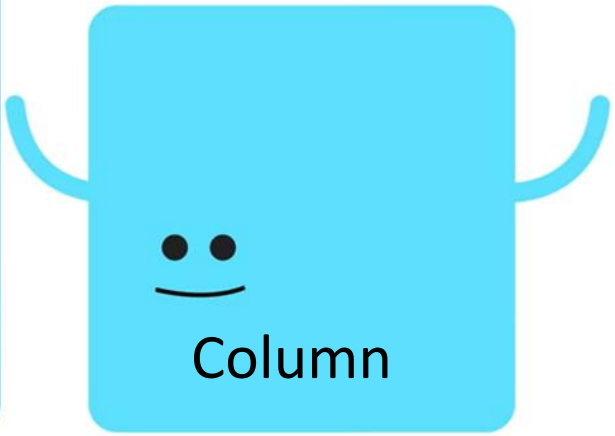
scaffold size constraint :

- **Height**: max. device height
- **Width**: max. device width



column size preferences :

- **Height**: as much as possible
- **Width**: as much as needed by children



RESULT

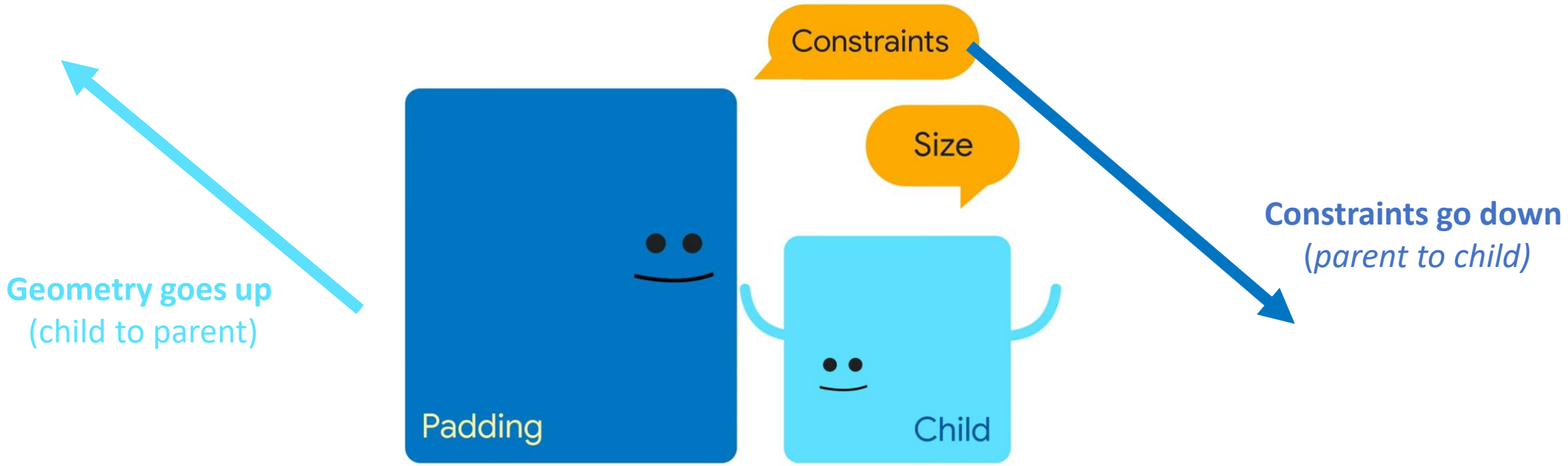


height: max. device
width: depends on children

EXPLAIN

Constraints go down. Sizes go up. Parent sets position

Flutter layout can't really be understood without knowing this rule



https://www.youtube.com/watch?v=jckqXR5CrPI

https://docs.flutter.dev/ui/layout/constraints

EXPLAIN

SingleChildScrollView

Wrap the column in a scrollable area !

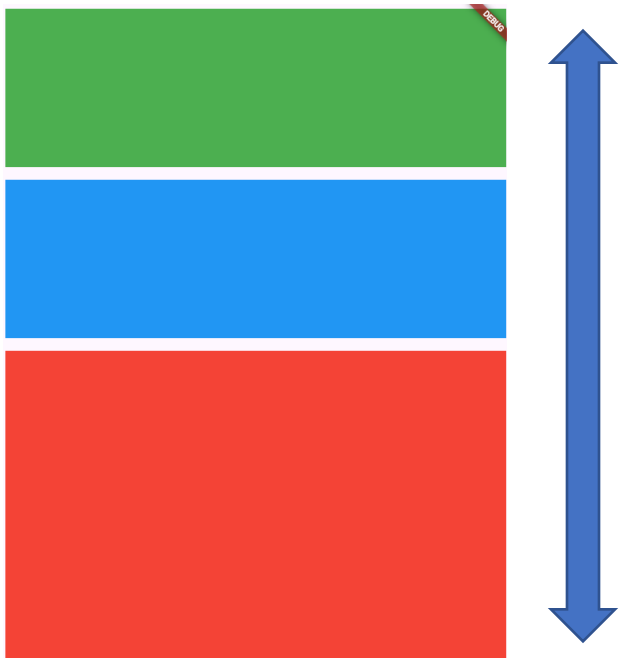
MORE INFO

```
body: Column(
    children: [ …],
),
```



Overflow !

```
body:
  SingleChildScrollView(
     child: Column(
          children: [ …],
  )
),
```



EXPLAIN

Understand the **Unbounded height** error !

MORE INFO

```
home: Scaffold(
          body: SingleChildScrollView(
            child: Column(
              children: [
                Expanded(
                  child: Padding(
                    child: Container(height: 200),
                  ),
                ),
              ],
            ),
          ),
)),
```

1 - If a Column is inside a **scrollable area**, its **height is unbounded**, meaning it can be as tall as necessary.

2 - However, when you use a **widget like Expanded inside the Column**, you're telling that widget to stretch and fill all the available space.

3 - since the parent doesn't have a set height, the **Expanded widget doesn't know how much space to fill !**

```
EXCEPTION CAUGHT BY RENDERING LIBRARY
The following assertion was thrown during performLayout():
RenderFlex children have non-zero flex but incoming height constraints are unbounded.
When a column is in a parent that does not provide a finite height constraint, for example if it is
in a vertical scrollable, it will try to shrink-wrap its children along the vertical axis. Setting a
flex on a child (e.g. using Expanded) indicates that the child is to expand to fill the remaining
space in the vertical direction.
These two directives are mutually exclusive. If a parent is to shrink-wrap its child, the child
cannot simultaneously expand to fit its parent.
Consider setting mainAxisSize to MainAxisSize.min and using FlexFit.loose fits for the flexible
children (using Flexible rather than Expanded). This will allow the flexible children to size
themselves to less than the infinite remaining space they would otherwise be forced to take, and
then will cause the RenderFlex to shrink-wrap the children rather than expanding to fit the maximum
constraints provided by the parent.
```

SOLUTION !

```
runApp(
  MaterialApp(
    home: Scaffold(
      body: SingleChildScrollView(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Padding(
              padding: const EdgeInsets.all(8.0),
              child: Container(color: Colors.green, height: 200),
            ), // Padding
            Flexible (
              fit: FlexFit.loose,
              child: Padding(
                padding: const EdgeInsets.all(8.0),
                child: Container(color: Colors.blue, height: 200),
              ), // Padding
            ), // Flexible
            Padding(
              padding: const EdgeInsets.all(8.0),
              child: Container(color: Colors.red, height: 100),
            ) // Padding
          ],
                ), // Column
      ), // SingleChildScrollView
    )), // Scaffold // MaterialApp
  );
}
```

Setting mainAxisSize to MainAxisSize.min, which will make the Column take only the space its children need.

Use Flexible instead of Expanded, and set FlexFit.loose, so the flexible children take only the space they need instead of trying to fill an undefined height.

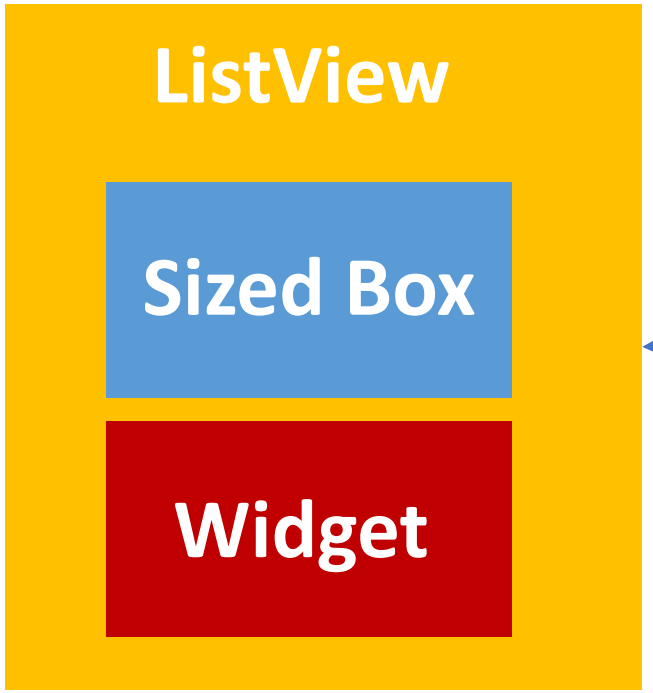
EXPLAIN

Create Empty Space

Padding or *Sized Box*




```
ListView(
    children: [
        Padding(
            padding: EdgeInsets.all(8.0),
            child: Text(“hello")),
),],)
```



Usually easier to understand..

```
ListView(
    children: [
        SizedBox(height: 20),
        Text(“hello"),
],)
```

Activity 1

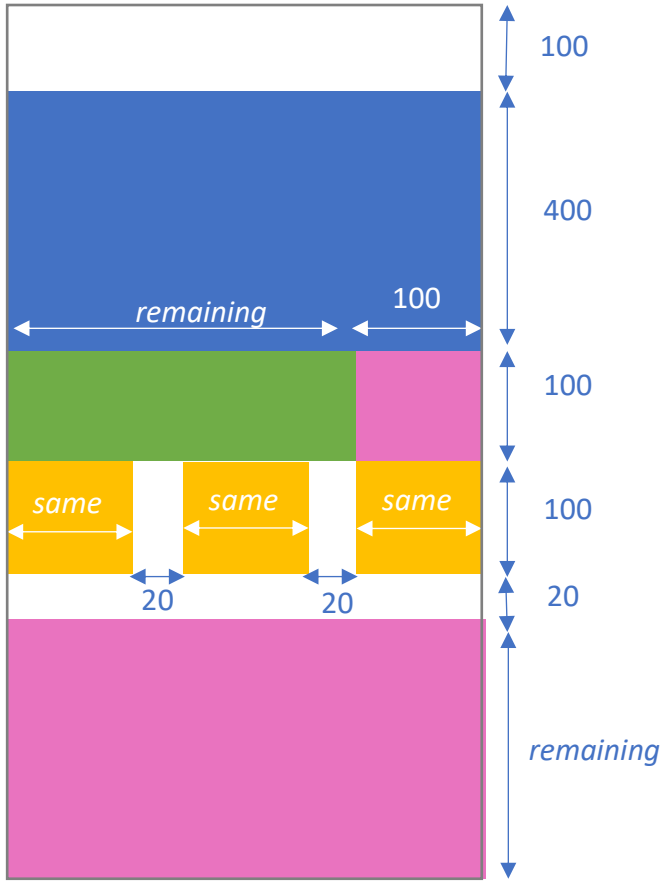
Build the bellow UI with the bellow widgets

Widgets:

- Column
- Row
- Expanded
- Container
- SizedBox

Configuration classes

- EdgeInsets
- CrossAxisAlignment
- MainAxisAlignment



Create your **PROFILE** page (free style !)

Your goal is to combine **all the widgets you have learned** and create a **welcome page of your choice**



STRUCTURE	Scaffold	Provides a basic structure for a mobile screen.
LAYOUT & ALIGNMENT	Row	Arranges widgets horizontally.
	Column	Arranges widgets vertically.
	Center	Aligns a child widget to the center.
	Expanded	Makes a child of a Row, Column, or Flex fill the available space.
	SingleChildScrollView	Makes a single child scrollable.
SPACING & STYLING	Container	A customizable box for layout and styling.
	SizedBox	Defines a fixed width or height, or adds space between widgets.
	Padding	Adds empty space around a widget's child.
CONTENT	Text	Displays text.

BONUS

Cards Exploration

Let s explore 3 kind of cards !

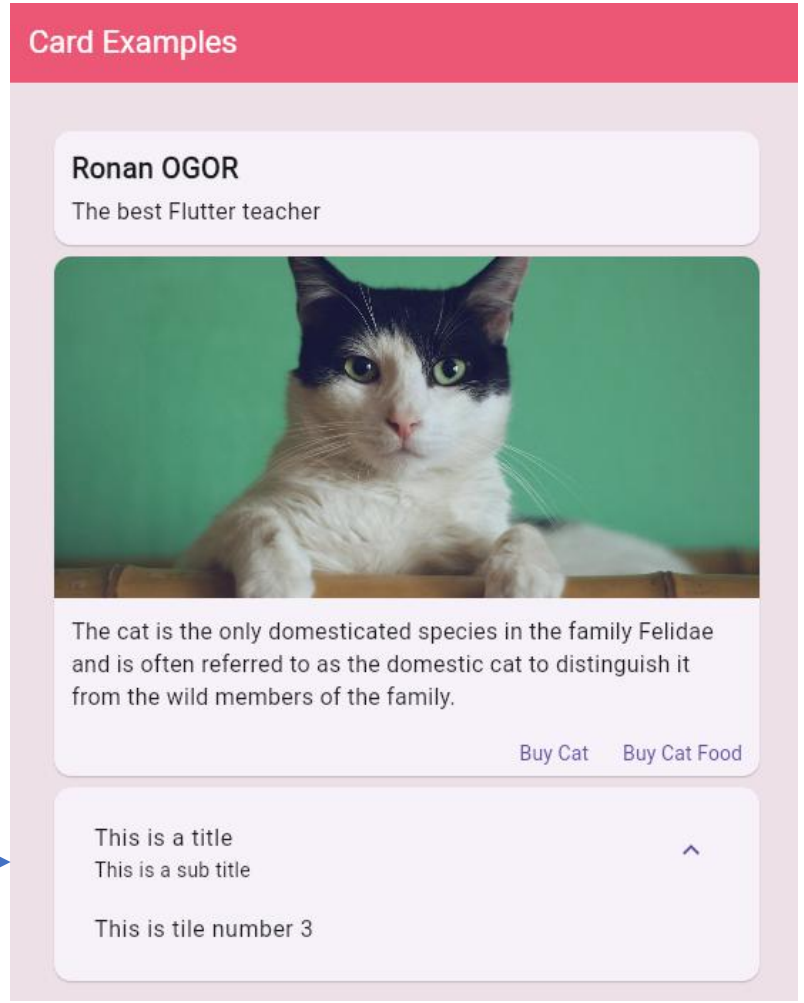
https://zapp.run/edit/cards-and-expanded-tiles-zh30060zh310?file=lib/main.dart

Try it

Create a `Card` with a `Column` of 2 Texts

Add an `Ink.image` and its related `InkWell`
Add a `OverflowBar` and 2 `TextButtons`

Add a `ExpansionTile` and its related `ListTile`



TODO AFTER

REFERENCES

Flutter Constraints
https://docs.flutter.dev/ui/layout/constraints

How Flutter layout its widgets?
https://www.youtube.com/watch?v=jckqXR5CrPI

Flutter Layout Widgets
https://docs.flutter.dev/ui/widgets/layout

Flutter Layouting tutorial
https://docs.flutter.dev/ui/layout

Row and Columns alignment
https://themobilecoder.com/axis-alignment-columns-rows-flutter/

Unbounded widgets
https://www.youtube.com/watch?v=uisLoOmtISk
https://www.youtube.com/watch?v=jckqXR5CrPI

Expanded Widget
https://www.youtube.com/watch?v=_rnZaagadyo

Flutter cards
https://api.flutter.dev/flutter/material/Card-class.html